

견적서의 나머지 항목

원칙 5 개로 다 못 줄인 비용은 어디 있나

캐싱 · 압축 · 컴팩션 · 서브에이전트 · 라우팅 · 배치

각 레버의 조건과 함정을 한 장씩 본다

레버는 곱해진다 — 우선순위는 뒤에서

캐시를 깨는 한 줄

\$10.05 (17.5% 더 지출)

맨 앞의 타임스탬프 · 요청 ID 는 매번 미스로 만든다

변하는 값은 반드시 뒤로 보낸다

미스 상태의 캐싱은 안 쓰느니만 못하다

압축과 캐싱은 대체재가 아니다

\$0.4014 → \$0.2007 (94% 절감)

그냥 캐싱 \$0.4014 vs 절반 요약 \$1.8000

지울 수 있는 반복은 지우고, 지울 수 없는 반복은 고정해 캐시에 태운다

압축률이 높을수록 조합의 이득도 커진다

긴 세션은 컴팩션으로 접는다

\$2.32 → \$0.93 (60% 절감)

10 턴마다 접으면 \$1.23 (47% 절감)

너무 자주 접으면 요약 호출이 이득을 먹는다

최적 주기는 세션 길이와 요약 비용으로 정해진다

툴 출력과 서버에이전트는 전체 비용으로 본다

\$5.6010 → \$2.7360

툴 읽기 누적 전송 1,680,000 토큰 (20 턴)

컴팩션 \$3.4425 · 서버에이전트 \$2.736

자식 호출 · 요약 비용까지 포함해 비교한다

모델 라우팅 · 쉬운 요청에 비싼 모델을 쓰지 않는다

전부 강 \$350 → 절반 \$184

절반만 라우팅해도 47% 절감

공격적 라우팅은 \$35.17 (90% 절감)

품질 측정 도구가 먼저다 — 임계값은 주기적으로 재보정

배치 API · 지연을 할인으로 바꾼다

50,000 건 \$675 → \$337.50

입력 · 출력 모두 50% 할인

24 시간 안에 끝나면 되는 잡만

야간 평가 · 대량 분류 · 리포트 생성 · 레포 전체 분석

레버는 곱해진다

85.825% (잔여 14%)

시맨틱 캐시 → 라우팅 → 캐싱 → verbosity/effort → 배치

잔여 비용에 순차 적용한 산술 결과

레버 간 중복 · 상호작용은 검증하지 않았다

캐싱과 개인정보

개인화 · 실시간 응답은 캐싱 금지

멀티턴은 캐시 키에 컨텍스트를 포함

시맨틱 캐시 임계값이 느슨하면 '비슷하지만 틀린 답'을 준다

삭제 · TTL 정책을 프롬프트 설계와 함께 정한다

작게 시작해서 자기 usage 로 확인한다

오늘 한 가지만 켜라 — 캐시

다음 주에 배수와 적중률을 다시 켜다

품질 게이트를 함께 둔다 — 절감만으로 대신하지 않는다

질문과 재현 결과는 저장소 이슈로 남긴다